CNP Business Flow Gaps

CNP Business Flow Gaps — Three Designs

Three items identified as missing from the seven core business flows: NDL check (Flow 1), Annual PY Rollover workflow (Flow 2), and FNS-640/905 reports (Flow 6).

1

NDL Check — National Disqualified List

Flow 1: Sponsor Onboarding & Application

Before a sponsor can be approved to participate in any federal nutrition program, USDA requires the state agency to verify the applicant is not on the National Disqualified List — a federal database of individuals and entities permanently barred from USDA food assistance programs due to fraud, misuse of funds, or serious program violations. An approved sponsor with an active NDL match is a federal compliance violation.

The NDL check must run at application approval time as a named condition hook on the `ApproveApplication` workflow transition. If a match is found, the transition is blocked and a review finding is created requiring the state agency to investigate before approval can proceed.

Integration design

USDA publishes the NDL as a searchable web resource. The platform integrates via the USDA FNS NDL lookup endpoint, checking three identifying fields: organization name, EIN (Employer Identification Number), and the names of key principals listed on the application (director, authorized representative).

Diagram 1 — NDL check in the application approval workflow

ApplicationUnderReviewApproveNDL Check HookNoNDLMatch conditionChecks name + EIN+ principal namesUSDA NDL APIfns.usda.gov/NDLBLOCKED — NdlCheckResults createdApprovedFreezeClaimLawsfiresno matchmatch found

Named hook: NoNDLMatch

```
-- WorkflowTransitionConditions row
TransitionCode  = 'ApproveApplication'
ConditionCode   = 'NoNDLMatch'
ConditionType   = 'PreCondition'
IsFederal       = true
FailureMessage  = 'Application cannot be approved: NDL match found.
Review required before proceeding.'

-- Hook implementation calls USDA NDL API and writes result
public class NoNDLMatchCondition : IWorkflowCondition {
```

```
  public async Task<bool> EvaluateAsync(WorkflowContext ctx) {
    var app = await _apps.GetAsync(ctx.EntityId);
    var result = await _ndlService.CheckAsync(app.OrgName, app.EIN,
app.Principals);
    await _ndlResults.WriteAsync(new NdlCheckResult {
      ApplicationId = ctx.EntityId, CheckDate = NOW(),
      MatchStatus = result.Status,  // NoMatch | PossibleMatch |
ConfirmedMatch
      MatchDetails = result.Matches
    });
    return result.Status == NdlMatchStatus.NoMatch;
  }
}
```

Tables (Applications domain, cnp_ops)

Table	Key columns	Notes
NdlCheckResults	ResultId PK, ApplicationId FK, CheckDate, MatchStatus (NoMatch|PossibleMatch|ConfirmedMatch), MatchDetails JSONB, CheckedBy (System|Manual)	Written by the NoNDLMatch hook on every ApproveApplication attempt. Retained permanently for audit.
NdlOverrideRequests	OverrideId PK, ResultId FK, RequestedBy FK, ReviewedBy FK, Disposition (Approved|Denied), Justification	When a PossibleMatch requires human review, this tracks the investigation outcome. A Denied disposition keeps the block active. An Approved disposition allows the approval to proceed with the override recorded.

Confirmed matches cannot be overriddenA ConfirmedMatch on the NDL is an absolute block — no override is possible. Only PossibleMatch results can go through the NdlOverrideRequests review process. This is enforced in code: the NoNDLMatch hook returns false for any ConfirmedMatch regardless of any NdlOverrideRequest outcome.

Annual Program Year Rollover

Flow 2: Annual Program Year Rollover — pre-populated renewal, checklist, attestation, state approval

Every July 1, the platform needs to roll every active sponsor's agreements forward into the new school year. A school district that was approved for NSLP SY2025-26 needs a new agreement for SY2026-27. Rather than making sponsors start from scratch, the system pre-populates a renewal with the prior year's data — sites, menus, claim history — and asks the sponsor to confirm it's still accurate, complete a checklist, and have an authorized director attest to the information.

This is different from a new application. The data model is the same (SponsorApplications, ProgramGroupYears) but the workflow is shorter and the initial state is system-generated rather than user-initiated.

Rollover workflow state machine

Diagram 2 — Annual rollover workflow states

AutoGeneratedHangfireOct 1, midnightChecklistSponsor verifiesprior-year dataAttestationDirector signselectronicallyStateReviewState approvesor requests changesApprovedNew PGY activeClaimLaws frozenState may waive

The Hangfire job

A scheduled Hangfire job fires at midnight UTC on October 1 for each active tenant. It is the only system that creates AutoGenerated renewal records — sponsors cannot initiate a rollover themselves.

```
// Hangfire job: ProgramYearRolloverJob
// Fires: 0 0 1 7 * (midnight UTC, October 1)
// Per tenant — one job execution per active state

public async Task ExecuteAsync(string tenantCode) {
  var newPGY = await _pgyService.CreateNewSchoolYear(tenantCode,
"SY2026-27");

  var activeSponsors = await
_sponsors.GetActiveEnrollments(tenantCode);

  foreach (var sponsor in activeSponsors) {
    // Copy prior-year application data as starting point
    var renewal = await _apps.CreateRenewalFromPriorYear(
      tenantCode, sponsor.SponsorId, newPGY.ProgramGroupYearId);

    // Set status = AutoGenerated, emit notification to sponsor
    await _workflow.TransitionAsync(renewal.ApplicationId,
      "RolloverAutoGenerate", "System");

    await _pyr.WriteAsync(new PYRolloverSponsor {
      TenantCode = tenantCode, SponsorId = sponsor.SponsorId,
      ApplicationId = renewal.ApplicationId, GeneratedAt = NOW()
```

```
    });
  }
  await _pyr.WriteJobSummary(tenantCode, activeSponsors.Count);
}
```

Tables (Applications domain, cnp_ops)

Table	Key columns	Notes
PYRolloverJobs	JobId PK, TenantCode, SchoolYear, StartedAt, CompletedAt, SponsorsProcessed, SponsorsErrored, Status	One row per tenant per July 1 (NSLP/SBP/CACFP) or January/February (SFSP); IsSFSP distinguishes the two. Tracks the batch job execution for monitoring and retry.
PYRolloverSponsors	RollId PK, JobId FK, SponsorId FK, ApplicationId FK, GeneratedAt, CurrentStatus, CompletedAt null	One row per sponsor per rollover. Tracks where each sponsor is in the workflow. State can filter by status to see who hasn't completed attestation.
RenewalChecklists	ChecklistId PK, ApplicationId FK, ItemCode, ItemDescription, CompletedBy FK null, CompletedAt null, Notes	Pre-populated checklist items from the prior year. Sponsor must mark each complete. Common items: verify site list, confirm FSMC contract if applicable, update meal price information.
RenewalAttestations	AttestationId PK, ApplicationId FK, AttestantUserId FK, AttestantTitle, SignedAt, IPAddress,	Electronic signature record. Immutable once created. Attests that the renewal data is accurate to the best of

	AttestationText (the legal statement agreed to)	the sponsor's knowledge.

State bulk approval States with hundreds of sponsors don't individually review every routine renewal. The StateReview state supports bulk approval: a state admin can select all renewals with no changed data, no open compliance findings, and no NDL concerns, and approve them in a single action. Bulk approval writes individual Approved state transitions for each — the workflow log shows who approved and when, preserving the audit trail even for bulk operations.

SFSP timing exceptionSFSP sponsors renew on a different cycle — summer program applications typically open February–April for the coming summer. SFSP rollover uses the same workflow and tables but fires on a different schedule and generates a SchoolYear of "Summer2027" rather than "SY2026-27". The IsSFSP flag on the PYRolloverJob row distinguishes the two cycles.

> ⚠ **Rate Availability Constraint**
> The rollover job (firing ~May 1) and rate insertion are independent operations. Rollover creates the new ProgramGroupYear and seeds application shells — it does not require rates to be present. Rates must be inserted into `ClaimRateAmounts` separately, before July 1, when USDA publishes (typically late June). A new PGY with no rates is valid; a claim submitted against it will fail rate validation until rates exist. Both operations must complete before claims can be processed for the new school year. Inserting rates before June 30 is an operational obligation, not just a data entry task.

FNS-640 and FNS-905 Reports

Flow 6: Disbursement of Funds — completing the FNS reporting catalog

The Reporting & Analytics domain currently covers FNS-10, FNS-44, and FNS-418. The spec also requires FNS-640 (Administrative Review Information) and FNS-905 (Direct Certification and Verification Summary). Both are annual reports submitted to USDA by each state agency.

FNS-640 — Child Nutrition Programs Administrative Review Information

FNS-640 is the annual summary of compliance review activity. Each state submits it to USDA showing how many reviews were conducted, what types of findings were identified, how many fiscal actions were assessed, and how much money was recovered. USDA uses it to evaluate whether state agencies are meeting their review cycle requirements across all programs.

Data sources (Snowflake)

Report section	Source tables	Notes
Reviews conducted by program type	vw_reviews_summary → AdminReview, ReviewType, ProgramCode	Count of completed reviews per program per SY
Findings by category	vw_compliance_findings → ReviewFindings, FindingCode	Grouped by FindingCategory (Fiscal, Operational, Eligibility)
Fiscal actions assessed	vw_fiscal_actions → FA1-FA4 tables, assessed amounts	Total dollars assessed by program and fiscal action type
Fiscal actions recovered	vw_repayments → RepaymentSchedules, ReceivedChecks	Total dollars actually collected within the reporting year
Serious deficiency proceedings	vw_serious_deficiencies → SeriousDeficiency schema	Count initiated, resolved, resulting in termination
Review cycle compliance	vw_review_cycles → RiskScores, ReviewCycles	% of sponsors reviewed within required cycle (3-year or annual for high-risk)

New Snowflake view: vw_fns640_summary

```
CREATE VIEW analytics.vw_fns640_summary AS
SELECT
  r.TenantCode,
  r.SchoolYear,
  r.ProgramCode,
  COUNT(*) AS ReviewsConducted,
  COUNT(CASE WHEN r.HasFindings THEN 1 END) AS ReviewsWithFindings,
  SUM(fa.AmountAssessed) AS TotalFiscalActionAssessed,
  SUM(rp.AmountRecovered) AS TotalFiscalActionRecovered,
  COUNT(sd.SerDefId) AS SeriousDeficienciesInitiated
FROM analytics.AdminReviews r
LEFT JOIN analytics.FiscalActions fa ON fa.ReviewId = r.ReviewId
LEFT JOIN analytics.RepaymentSummary rp ON rp.ReviewId = r.ReviewId
LEFT JOIN analytics.SeriousDeficiencies sd ON sd.ReviewId = r.ReviewId
GROUP BY r.TenantCode, r.SchoolYear, r.ProgramCode
```

FNS-905 — Direct Certification and Verification Summary

FNS-905 is the annual report on direct certification rates and eligibility verification activity. States submit it to show USDA how many students were directly certified (automatically, without household income applications), how many were included in the annual verification sample, and what the verification outcomes were. USDA uses it to evaluate whether states are meeting minimum DC rate targets and running compliant verification processes.

Data sources (Snowflake)

Report section	Source tables	Notes
DC certifications by source	vw_dc_summary → Core2_DirectCertMatches, CertSource (SNAP/Medicaid/TANF/FDPIR)	Total students DC certified by source program
DC rate (% of enrolled students)	vw_dc_summary joined to enrollment counts	DC certifications ÷ total enrolled, per state
Verification sample size	vw_verification → AnnualVerificationForms, SampleCount	Required sample = 3% of approved free/reduced applicants (non-CEP)
Verification outcomes	vw_verification → AnnualVerificationForms, VerifiedCount, ErrorCount	How many verified, how many changed eligibility, how many errors found
Error rate	vw_verification → ErrorCount ÷ SampleCount	USDA tracks state error rates over time. High error rates trigger additional oversight.

New Snowflake view: vw_fns905_summary

```
CREATE VIEW analytics.vw_fns905_summary AS
SELECT
  d.TenantCode,
  d.SchoolYear,
  COUNT(*) AS TotalDCCertifications,
  COUNT(CASE WHEN d.CertSource = 'SNAP' THEN 1 END) AS
SNAPCertifications,
```

```
  COUNT(CASE WHEN d.CertSource = 'Medicaid' THEN 1 END) AS MedicaidCertifications,
  COUNT(CASE WHEN d.CertSource = 'TANF' THEN 1 END) AS TANFCertifications,
  e.TotalEnrolled,
  ROUND(COUNT(*) * 100.0 / NULLIF(e.TotalEnrolled, 0), 2) AS DCRatePct,
  v.SampleCount AS VerificationSampleSize,
  v.VerifiedCount,
  v.ErrorCount,
  ROUND(v.ErrorCount * 100.0 / NULLIF(v.SampleCount, 0), 2) AS ErrorRatePct
FROM analytics.DirectCertMatches d
JOIN analytics.EnrollmentSummary e USING (TenantCode, SchoolYear)
LEFT JOIN analytics.VerificationSummary v USING (TenantCode, SchoolYear)
GROUP BY d.TenantCode, d.SchoolYear, e.TotalEnrolled, v.SampleCount, v.VerifiedCount, v.ErrorCount
```

Query catalog additions

Template ID	Plain-English intent	View	Available to
`FNS640ReviewSummary`	"How many compliance reviews did we complete?" / "Show me our FNS-640 data"	vw_fns640_summary	Director, Finance
`FNS905DCVerification`	"What's our direct certification rate?" / "Show me our FNS-905 verification summary"	vw_fns905_summary	Director, Finance

FNS-640 and FNS-905 both feed the compliance risk scoring AI The risk scoring model uses DC match rate as a signal (vw_dc_summary, which feeds FNS-905) and review cycle compliance as a signal (vw_review_cycles, which feeds FNS-640). Adding these two views to the reporting catalog means the same data that drives AI-generated risk alerts is also what populates the federal submissions. One source of truth.